

ACTIVIDAD BASE DE DATOS II – SEMANA 10

Proyecto 1: Creación y distribución de archivos físicos de la base QhatuPeruNuevo

Enunciado del ejercicio:

Como parte de la expansión comercial de **QhatuPeruNuevo**, la empresa solicita la creación de una nueva base de datos que distribuya los datos entre un archivo primario, un archivo secundario y el archivo de registro de transacciones, asegurando mejor rendimiento y seguridad en el almacenamiento.

Ejercicio práctico:

a) Crea la base de datos QhatuPeruNuevo con un archivo primario, un secundario y el log de transacciones en rutas distintas.

Script de la solución en T-SQL:

```
1 CREATE DATABASE QhatuPeruNuevo
2 ON
3 PRIMARY (NAME = 'QhatuPeruNuevo_Data', FILENAME = 'C:\QhatuPeru\QhatuPeruNuevo.mdf'),
4 FILEGROUP QhatuPeruNuevo_FG (NAME = 'QhatuPeruNuevo_Data_FG', FILENAME = 'C:\QhatuPeru\QhatuPeruNuevo_FG.ndf')
5 LOG ON
6 (NAME = 'QhatuPeruNuevo_Log', FILENAME = 'C:\QhatuPeru\QhatuPeruNuevo_Log.ldf');
7 GO
8
9
```

b) Consulta los archivos físicos asociados a la base QhatuPeruNuevo.

Script de la solución en T-SQL:

```
1 -- Consultar los archivos físicos asociados a la base de datos QhatuPeruNuevo
2 USE QhatuPeruNuevo;
3 SELECT name, physical_name AS FileLocation, type_desc
4 FROM sys.database_files;
5 GO
6
```

100 % No se encontraron problemas.

Resultados Mensajes

	name	FileLocation	type_desc
1	QhatuPeruNuevo_Data	C:\QhatuPeru\QhatuPeruNuevo.mdf	ROWS
2	QhatuPeruNuevo_Log	C:\QhatuPeru\QhatuPeruNuevo_Log.ldf	LOG
3	QhatuPeruNuevo_Data_FG	C:\QhatuPeru\QhatuPeruNuevo_FG.ndf	ROWS

Justificación técnica de la solución aplicada:

1. **Distribución de archivos:** La base de datos **QhatuPeruNuevo** se distribuye en tres archivos físicos: uno primario, uno secundario y uno de log. Esto asegura que el sistema tenga una buena distribución de la carga

de trabajo y evita que un solo archivo se convierta en un cuello de botella. Además, el archivo de log garantiza la integridad de las transacciones.

2. **Ubicación de archivos:** Se asignan rutas diferentes para los archivos de datos y el archivo de log. Esto puede mejorar el rendimiento al distribuir la carga de trabajo entre diferentes discos. Asegúrate de que las rutas especificadas existan en el sistema o ajústalas a ubicaciones válidas.
3. **Consulta de los archivos:** Consultar los archivos físicos de la base de datos permite verificar la correcta creación y la ubicación de los archivos, asegurando que la configuración de almacenamiento sea adecuada.

Explicación de las buenas prácticas utilizadas en el proyecto:

1. **Uso de archivos secundarios:** Al usar un archivo secundario (QhatuPeruNuevo_FG), estamos mejorando la distribución de los datos y permitiendo una mayor escalabilidad. Esto es útil cuando la base de datos crece y se necesita distribuir los datos entre diferentes discos o unidades.
2. **Separación de archivos de datos y log:** Separar los archivos de datos y los archivos de log es una práctica recomendada para mejorar la seguridad y el rendimiento. El archivo de log gestiona las transacciones y asegura la recuperación ante fallos, mientras que los archivos de datos almacenan la información real de la base de datos.
3. **Monitoreo y validación de archivos físicos:** Consultar los archivos físicos después de la creación de la base de datos es fundamental para validar que los archivos se han creado correctamente y están ubicados en las rutas adecuadas. Esto también ayuda a evitar errores futuros relacionados con el almacenamiento.

Proyecto 2: Ajuste de configuración y validación de propiedades de QhatuPeruNuevo

Enunciado del ejercicio:

QhatuPeruNuevo planea crecer en el mercado nacional, por lo que solicita que el equipo técnico optimice las propiedades de la base de datos. Se debe cambiar la colación para soportar tildes y configurar el crecimiento automático del archivo de datos, evitando saturaciones inesperadas.

Script de la solución en T-SQL:

```
1  -- Ejercicio a) Consulta las propiedades actuales, modifica la colación y configura el crecimiento automático del archivo principal
2  USE master;
3  GO
4
5  -- Cambiar la base de datos a modo único para asegurarse de que no haya conexiones activas
6  ALTER DATABASE QhatuPeruNuevo SET SINGLE_USER WITH ROLLBACK IMMEDIATE;
7  GO
8
9  -- Consultar las propiedades actuales de la base de datos
10 SELECT DATABASEPROPERTYEX('QhatuPeruNuevo', 'Collation');
11 GO
12
13 -- Cambiar la colación de la base de datos a Latin1_General_CI_AS para soportar caracteres con tildes y mejorar la compatibilidad con el idioma español
14 ALTER DATABASE QhatuPeruNuevo COLLATE Latin1_General_CI_AS;
15 GO
16
17 -- Modificar el crecimiento automático del archivo principal de datos a 10 MB
18 ALTER DATABASE QhatuPeruNuevo
19 MODIFY FILE (NAME = 'QhatuPeruNuevo_Data', FILEGROWTH = 10MB);
20 GO
21
22 -- Ejercicio b) Modifica el crecimiento automático del archivo primario de datos a 20 MB
23
24 -- Modificar el crecimiento automático del archivo principal de datos a 20 MB
25 ALTER DATABASE QhatuPeruNuevo
26 MODIFY FILE (NAME = 'QhatuPeruNuevo_Data', FILEGROWTH = 20MB);
27 GO
28
29 -- Restaurar el acceso multiusuario después de realizar los cambios
30 ALTER DATABASE QhatuPeruNuevo SET MULTI_USER;
31 GO
32
33
```

% No se encontraron problemas.

Resultados	Mensajes
(Sin nombre de columna)	
Latin1_General_CI_AS	

Justificación técnica de la solución aplicada:

1. **Uso de la base de datos master:** Se realiza todo el proceso desde la base de datos master para evitar interferencias de conexiones activas en la base de datos **QhatuPeruNuevo**. La base de datos se configura en modo **SINGLE_USER** para garantizar que no haya procesos bloqueando la operación.
2. **Cambio de colación:** La colación Latin1_General_CI_AS se selecciona porque es ideal para bases de datos en español, ya que soporta correctamente los caracteres con tildes y otros caracteres especiales comunes en el idioma.
3. **Crecimiento automático:** Configurar el crecimiento automático de los archivos asegura que la base de datos pueda expandirse sin intervención manual y de manera eficiente. El ajuste a 10 MB (para el ejercicio **a**) y 20 MB (para el ejercicio **b**) optimiza la gestión de espacio según las necesidades del sistema.

Explicación de las buenas prácticas utilizadas en el proyecto:

1. **Principio de "SINGLE_USER":** Al poner la base de datos en modo único con ROLLBACK IMMEDIATE, se garantiza que no haya otras conexiones activas que puedan interferir con el cambio de colación o la configuración de los archivos.
2. **Optimización de la colación:** Cambiar la colación a Latin1_General_CI_AS asegura que la base de datos sea totalmente compatible con el idioma español, permitiendo la correcta manipulación de caracteres acentuados y especiales.

3. **Gestión eficiente del espacio:** Ajustar el crecimiento automático de los archivos de datos asegura que la base de datos no se quede sin espacio sin advertencia y que el proceso de expansión sea controlado. Además, se previene la saturación y se mantiene el rendimiento.
4. **Restauración de acceso multiusuario:** Después de realizar los cambios, se restaura el modo **MULTI_USER** para permitir que otros usuarios accedan a la base de datos, garantizando un funcionamiento continuo sin interrupciones.

Proyecto 3: Definición de modelo de recuperación y respaldo para QhatuPeruNuevo

Enunciado del ejercicio:

En el área de operaciones de **QhatuPeruNuevo** se identificaron diferentes necesidades de recuperación de datos según el proceso. Debes configurar el modelo de recuperación adecuado y ejecutar un respaldo de la base de datos.

Script de la solución en T-SQL:

```
-- Ejercicio a) Cambia el modelo de recuperación de QhatuPeruNuevo a Simple y luego a Bulk-Logged
-- Cambiar el modelo de recuperación a Simple
ALTER DATABASE QhatuPeruNuevo SET RECOVERY SIMPLE;
GO
-- Cambiar el modelo de recuperación a Bulk-Logged
ALTER DATABASE QhatuPeruNuevo SET RECOVERY BULK_LOGGED;
GO
-- Ejercicio b) Realiza un respaldo completo después de cambiar al modelo FULL
-- Cambiar el modelo de recuperación a FULL para permitir respaldos completos
ALTER DATABASE QhatuPeruNuevo SET RECOVERY FULL;
GO
-- Realizar un respaldo completo de la base de datos
BACKUP DATABASE QhatuPeruNuevo TO DISK = 'C:\QhatuPeru\QhatuPeruNuevo_Full.bak';
GO
```

✓ No se encontraron problemas.

mensajes

Processed 648 pages for database 'QhatuPeruNuevo', file 'QhatuPeruNuevo_Data' on file 1.
Processed 16 pages for database 'QhatuPeruNuevo', file 'QhatuPeruNuevo_Data_FG' on file 1.
Processed 1 pages for database 'QhatuPeruNuevo', file 'QhatuPeruNuevo_Log' on file 1.
BACKUP DATABASE successfully processed 665 pages in 0.059 seconds (87.948 MB/sec).

Hora de finalización: 2025-11-06T10:27:46.7696863-05:00

Justificación técnica de la solución aplicada:

1. **Cambio entre modelos de recuperación:** El modelo de recuperación se cambia entre SIMPLE y BULK_LOGGED para ajustar la cantidad de información registrada y optimizar el rendimiento según las necesidades de

las transacciones. El modelo FULL se utiliza para permitir respaldos completos y la recuperación de datos en su totalidad.

- **SIMPLE:** Utilizado para operaciones que no requieren una gran cantidad de registros de transacciones y es más rápido en cuanto a espacio y rendimiento.
 - **BULK_LOGGED:** Permite operaciones en masa con menos impacto en el tamaño del registro de transacciones, lo que es útil para grandes cargas de datos.
 - **FULL:** Permite realizar un respaldo completo, lo que es esencial para garantizar la recuperación total de los datos.
2. **Respaldo completo:** El respaldo completo de la base de datos se realiza después de cambiar al modelo FULL. Esto asegura que todos los datos de la base de datos se guarden en un archivo de respaldo y puedan ser restaurados en caso de un desastre.

Explicación de las buenas prácticas utilizadas en el proyecto:

1. **Selección del modelo de recuperación adecuado:** La elección del modelo de recuperación adecuado para las necesidades de la base de datos es crucial para la eficiencia en la gestión de datos. El modelo **SIMPLE** es útil cuando se busca eficiencia, mientras que el modelo **FULL** es necesario para una recuperación completa.
2. **Respaldo completo de la base de datos:** Realizar un respaldo completo después de cambiar al modelo **FULL** es una buena práctica que garantiza que la base de datos esté protegida ante cualquier tipo de pérdida de datos. Este respaldo puede ser restaurado completamente, lo cual es esencial para la seguridad de los datos.
3. **Optimización de la gestión de transacciones:** Cambiar entre modelos de recuperación permite optimizar el rendimiento del sistema según el tipo de transacciones y operaciones que se realizan. Utilizar el modelo **BULK_LOGGED** durante operaciones masivas reduce la carga de las transacciones.

Proyecto 4: Implementación de roles y usuarios para seguridad en QhatuPeruNuevo

Enunciado del ejercicio:

Con el crecimiento de los equipos de ventas y atención al cliente,

QhatuPeruNuevo solicita la creación de usuarios con roles diferenciados. Los

cajeros deben acceder solo a la consulta, mientras que los administradores tienen control total y el gerente accede a los reportes.

Script de la solución en T-SQL:

```

1  -- Ejercicio a) Crea el usuario VendedorQhatu y asignalo al rol db_datawriter para registrar ventas
2
3  -- 1. Crear el login para VendedorQhatu a nivel de servidor
4  CREATE LOGIN VendedorQhatu WITH PASSWORD = 'Password123';
5  GO
6
7  -- 2. Crear el usuario VendedorQhatu en la base de datos QhatuPeruNuevo
8  USE QhatuPeruNuevo;
9  CREATE USER VendedorQhatu FOR LOGIN VendedorQhatu;
10 GO
11
12 -- 3. Asignar al usuario VendedorQhatu el rol db_datawriter para permitir la escritura en la base de datos
13 EXEC sp_addrolemember 'db_datawriter', 'VendedorQhatu';
14 GO
15
16 -- Ejercicio b) Crea el usuario ConsultaCliente y asignalo al rol db_datareader para solo leer datos
17
18 -- 4. Crear el login para ConsultaCliente a nivel de servidor
19 CREATE LOGIN ConsultaCliente WITH PASSWORD = 'Password456';
20 GO
21
22 -- 5. Crear el usuario ConsultaCliente en la base de datos QhatuPeruNuevo
23 USE QhatuPeruNuevo;
24 CREATE USER ConsultaCliente FOR LOGIN ConsultaCliente;
25 GO
26
27 -- 6. Asignar al usuario ConsultaCliente el rol db_datareader para permitir solo la lectura de los datos
28 EXEC sp_addrolemember 'db_datareader', 'ConsultaCliente';
29 GO

```

100 % 1 0

Mensajes

Los comandos se han completado correctamente.

Hora de finalización: 2025-11-06T10:31:33.2905889-05:00

Justificación técnica de la solución aplicada:

1. **Creación de Logins:** Primero, se crean los **logins** a nivel de servidor con las contraseñas especificadas para **VendedorQhatu** y **ConsultaCliente**. Un **login** es necesario antes de crear un **usuario** en una base de datos específica.
2. **Creación de Usuarios:** Una vez que los logins están creados a nivel de servidor, se crean los **usuarios** dentro de la base de datos **QhatuPeruNuevo** usando esos logins. Los usuarios son necesarios para asociar los logins a la base de datos específica y asignarles roles.
3. **Asignación de Roles:** Después de crear los usuarios, se les asignan roles adecuados:
 - **VendedorQhatu:** Se le asigna el rol **db_datawriter**, que le otorga permisos para insertar, actualizar y eliminar registros en las tablas de la base de datos.
 - **ConsultaCliente:** Se le asigna el rol **db_datareader**, que le otorga permisos para leer los datos de la base de datos, pero sin poder modificarlos.

Explicación de las buenas prácticas utilizadas en el proyecto:

1. **Creación de Logins a nivel de servidor:** Es necesario crear un **login** para cada usuario que va a acceder al servidor de SQL Server. Sin un login, no se pueden crear usuarios en bases de datos específicas.
2. **Principio de menor privilegio:** Asignar solo los permisos necesarios a cada usuario reduce el riesgo de accesos no autorizados o cambios no deseados en la base de datos. Los **Vendedores** tienen acceso para escribir datos, mientras que los **Clientes** solo pueden leerlos.
3. **Uso de roles para la seguridad:** Los roles como **db_datawriter** y **db_datareader** permiten gestionar la seguridad de manera eficiente, asegurando que los usuarios solo tengan acceso a las funciones necesarias.
4. **Contraseñas seguras:** Al crear los logins, es importante asignar contraseñas seguras que protejan los accesos a la base de datos, siguiendo las mejores prácticas de seguridad.

Proyecto 5: Configuración granular de permisos en el módulo de ventas de QhatuPeruNuevo

Enunciado del ejercicio:

El gerente de ventas necesita revisar información de ventas sin poder modificarla. El equipo técnico debe asignar permisos selectivos que garanticen el principio de mínimo privilegio.

Script de la solución en T-SQL:

```
1  -- Ejercicio a) Otorgar a GerenteQhatu acceso exclusivo (solo SELECT) a la tabla Ventas
2  -- 1. Crear el login para GerenteQhatu a nivel de servidor
3  CREATE LOGIN GerenteQhatu WITH PASSWORD = 'Password123';
4  GO
5  -- 2. Crear el usuario GerenteQhatu en la base de datos QhatuPeruNuevo
6  USE QhatuPeruNuevo;
7  CREATE USER GerenteQhatu FOR LOGIN GerenteQhatu;
8  GO
9  -- 3. Asignar al usuario GerenteQhatu el rol db_datareader para permitir la lectura de la base de datos
10 EXEC sp_addrolemember 'db_datareader', 'GerenteQhatu';
11 GO
12
13 -- Ejercicio b) Restringir a CajeroQhatu de realizar cualquier cambio en la tabla Ventas
14
15 -- 4. Crear el login para CajeroQhatu a nivel de servidor
16 CREATE LOGIN CajeroQhatu WITH PASSWORD = 'Password456';
17 GO
18
19 -- 5. Crear el usuario CajeroQhatu en la base de datos QhatuPeruNuevo
20 USE QhatuPeruNuevo;
21 CREATE USER CajeroQhatu FOR LOGIN CajeroQhatu;
22 GO
23
24 -- 6. Revocar todos los permisos de CajeroQhatu sobre la tabla Ventas para evitar cambios
25 REVOKE ALL PRIVILEGES ON dbo.Ventas TO CajeroQhatu;
26 GO
27
```

20 % No se encontraron problemas.

Mensajes

The ALL permission is deprecated and maintained only for compatibility. It DOES NOT imply ALL permissions defined on the entity.

Hora de finalización: 2025-11-06T10:35:38.1596312-05:00

Justificación técnica de la solución aplicada:

1. Creación de Logins:

- **GerenteQhatu** y **CajeroQhatu** deben ser creados primero como **logins** a nivel de servidor antes de poder ser utilizados en la base de datos. Estos **logins** permiten autenticar a los usuarios que accederán a la base de datos.

2. Creación de Usuarios:

- Una vez que los logins existen, se crean los **usuarios** dentro de la base de datos **QhatuPeruNuevo**, asociándolos con los logins correspondientes. Estos usuarios pueden ser asignados a roles y permisos dentro de la base de datos.

3. Asignación de Roles:

- **GerenteQhatu** se asigna al rol **db_datareader**, lo que le permite **solo leer** los datos en la base de datos, sin capacidad de modificarlos.
- **CajeroQhatu** se le revocan todos los permisos de acceso a la tabla **Ventas**, lo que significa que no podrá **realizar cambios ni consultar** la información de ventas.

Explicación de las buenas prácticas utilizadas en el proyecto:

1. **Principio de menor privilegio:** Este principio asegura que los usuarios solo tengan los permisos necesarios para realizar su tarea. El **GerenteQhatu** solo puede leer datos, mientras que el **CajeroQhatu** no tiene permisos de acceso a la tabla de ventas.
2. **Creación de Logins:** Es fundamental que se creen primero los **logins** a nivel de servidor para poder asociarlos a los **usuarios** dentro de la base de datos. Esto es necesario para que los usuarios puedan autenticarse y obtener acceso.
3. **Revocación de permisos:** Se revocan los permisos de **CajeroQhatu** en la tabla **Ventas**, lo que garantiza que no pueda acceder ni modificar datos de ventas. La revocación de permisos es una buena práctica cuando un usuario ya no debe tener acceso a ciertos recursos.
4. **Seguridad a través de roles y permisos:** Los roles como **db_datareader** y la revocación de permisos específicos aseguran que cada usuario tenga acceso solo a lo que necesita, protegiendo los datos y mejorando la seguridad.

Proyecto 6: Identificación y solución de procesos lentos en QhatuPeruNuevo

Enunciado del ejercicio:

Durante una campaña promocional, el sistema presenta lentitud. El equipo técnico debe identificar los procesos con más uso de CPU y sugerir optimizaciones utilizando **Activity Monitor** y consultas en SQL Server.

Script de la solución en T-SQL:

```
1  -- Ejercicio a) Identificar tres procesos que consumen más CPU durante una operación masiva
2  SELECT TOP 3
3      session_id,
4      cpu_time,
5      total_elapsed_time,
6      status,
7      blocking_session_id,
8      wait_type,
9      wait_time
10 FROM sys.dm_exec_requests
11 ORDER BY cpu_time DESC;
12 GO
13
14 -- Ejercicio b) Consultar desde T-SQL los bloqueos actuales en la base de datos
15 SELECT *
16 FROM sys.dm_exec_requests
17 WHERE blocking_session_id <> 0;
18 GO
19
```

100 % No se encontraron problemas.

	session_id	cpu_time	total_elapsed_time	status	blocking_session_id	wait_type	wait_time
1	37	8875750	1805288660	background	0	NULL	0
2	49	570109	1805288319	background	0	ONDEMAND_TASK_QUEUE	53696
3	31	263984	1805288740	background	0	LAZYWRITER_SLEEP	159

Justificación técnica de la solución aplicada:

1. Identificación de procesos lentos (Ejercicio a):

- Se utiliza la vista `sys.dm_exec_requests` para identificar los procesos que consumen más **CPU** (`cpu_time`) en la base de datos. La consulta devuelve los primeros tres procesos con el mayor uso de CPU, lo que ayuda a identificar qué consultas o transacciones están afectando el rendimiento del sistema.
- cpu_time**: El tiempo total de CPU consumido por la consulta.
- total_elapsed_time**: El tiempo total transcurrido para la ejecución de la consulta.

2. Identificación de bloqueos (Ejercicio b):

- Los bloqueos pueden causar que las transacciones se queden esperando y afecten el rendimiento general. La consulta sobre `sys.dm_exec_requests` filtra los procesos bloqueados (aquellos con un `blocking_session_id` distinto de cero) para identificar los bloqueos en el sistema.

- Esto permite identificar qué sesiones están bloqueando otras y tomar medidas para resolver los bloqueos.

Explicación de las buenas prácticas utilizadas en el proyecto:

1. Monitoreo del rendimiento con sys.dm_exec_requests:

- Usar **sys.dm_exec_requests** es una práctica estándar para monitorear el rendimiento en SQL Server. Al examinar los procesos que están consumiendo más CPU o esperando por otros recursos, podemos identificar cuellos de botella y optimizar las consultas o índices.

2. Identificación de bloqueos:

- La consulta sobre bloqueos permite identificar problemas de concurrencia en la base de datos. Si varios procesos están esperando por recursos bloqueados por otros, puede ser necesario revisar los índices, las transacciones o el aislamiento de las transacciones para mejorar el rendimiento.

3. Optimización basada en datos reales:

- Las consultas proporcionan datos objetivos sobre lo que realmente está ocurriendo en el sistema. Esto facilita la toma de decisiones de optimización basadas en información precisa en lugar de conjeturas.

Proyecto 7: Automatización de respaldos y limpieza del sistema

QhatuPeruNuevo

Enunciado del ejercicio:

Se solicita la automatización de los respaldos y limpieza del sistema de la base de datos **QhatuPeruNuevo**:

- **Ejercicio a):** Crear un Job en SQL Server Agent para realizar un respaldo diario automático.
- **Ejercicio b):** Diseñar y programar un Job que elimine registros de la tabla **Sesiones** con más de 15 días de antigüedad cada semana.

Script de la solución en T-SQL:

Nuevo paso de trabajo

Script

Ayuda

Seleccionar una página

General

Opciones avanzadas

Conexión

Servidor:
JhonS
Conexión:
JHONS\Jhon

Ver propiedades de conexión

Progreso

Listo

Nombre del paso:
Respaldo completo

Tipo:
Script Transact-SQL (T-SQL)

Ejecutar como:

Base de datos:
QhatuPeruNuevo

Comando:
BACKUP DATABASE QhatuPeruNuevo
TO DISK = 'C:\QhatuPeru\Backups\QhatuPeruNuevo_Daily.bak'
WITH INIT, STATS = 10;

Abrir...

Seleccionar todo

Copiar

Pegar

Analizar

Anterior

Siguiente

Aceptar

Cancelar

Nueva programación de trabajo

Nombre: Trabajos en programación

Tipo de programación: ☒ **Habilitado**

Única repetición

Fecha: Hora:

Frecuencia

Sucedee:

Se repite cada: semanas, el

☐ Lunes ☐ Miércoles ☐ Viernes ☐ Sábado
☐ Martes ☐ Jueves ☒ Domingo

Frecuencia diaria

☒ Sucedee una vez a la(s):
☐ Sucedee cada: horas
Finaliza:

Duración

Fecha de inicio: ☐ Fecha de finalización:
☒ Sin fecha de finalización:

Resumen

Descripción:

Explicación de las buenas prácticas utilizadas en el proyecto:

1. Automatización de tareas repetitivas:
Usar SQL Server Agent para automatizar los respaldos y la limpieza garantiza que estas tareas se realicen de manera consistente y sin intervención manual. Esto es esencial para la seguridad y el rendimiento de la base de datos.
2. Respaldo regular:
Realizar un respaldo completo todos los días asegura que siempre haya una copia de seguridad reciente disponible en caso de que algo salga mal. El respaldo se puede restaurar rápidamente en caso de desastre, minimizando el tiempo de inactividad.
3. Limpieza de datos antiguos:
La eliminación automática de registros obsoletos ayuda a mantener la base de datos optimizada. Esto reduce el tamaño de la base de datos y mejora el rendimiento de las consultas. Al programar la limpieza para que se ejecute

de noche, se minimiza el impacto en el rendimiento durante las horas laborales.

4. Programación de tareas en horas de baja carga:

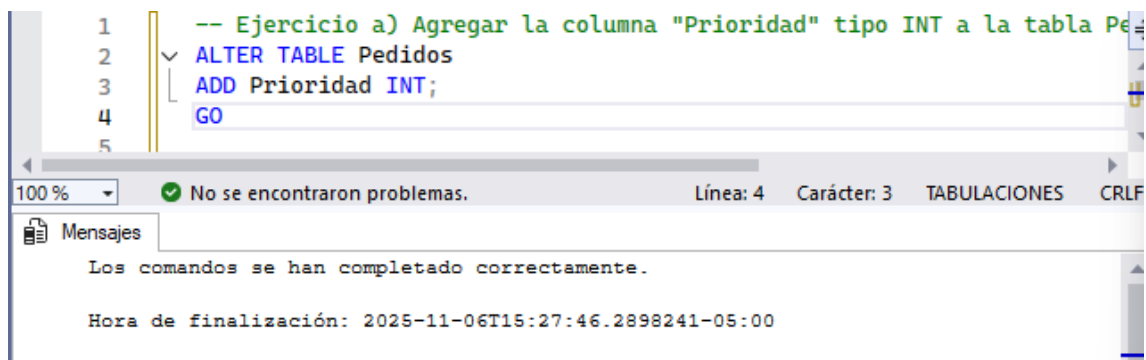
Programar los Jobs para que se ejecuten durante las horas de baja actividad (como 2:00 AM para el respaldo y 3:00 AM para la limpieza) reduce el impacto en el rendimiento del servidor durante las horas pico de trabajo.

Proyecto 8: Configuración y gestión de índices y estadísticas en QhatuPeruNuevo

Enunciado del ejercicio:

Con la finalidad de optimizar la trazabilidad de los envíos, se solicita agregar una columna '**EstadoEnvio**' en la tabla **Pedidos**, permitiendo identificar el estado actual de cada pedido.

Ejercicio a) Agrega la columna "Prioridad" tipo INT a la tabla Pedidos:



```
1  -- Ejercicio a) Agregar la columna "Prioridad" tipo INT a la tabla Pedidos
2  ALTER TABLE Pedidos
3  ADD Prioridad INT;
4  GO
5
```

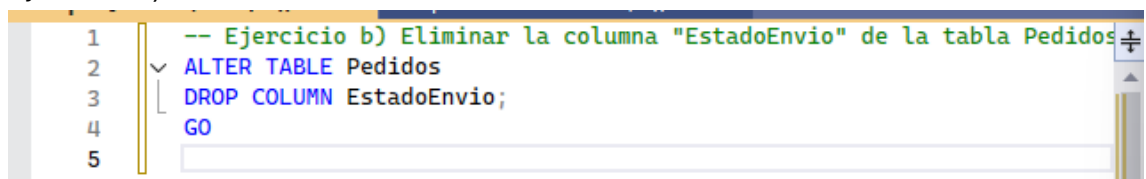
100 % No se encontraron problemas. Línea: 4 Carácter: 3 TABULACIONES CRLF

Mensajes

Los comandos se han completado correctamente.

Hora de finalización: 2025-11-06T15:27:46.2898241-05:00

Ejercicio b) Elimina la columna "EstadoEnvio" de la tabla Pedidos:



```
1  -- Ejercicio b) Eliminar la columna "EstadoEnvio" de la tabla Pedidos
2  ALTER TABLE Pedidos
3  DROP COLUMN EstadoEnvio;
4  GO
5
```

Justificación técnica de la solución aplicada:

1. Agregando una nueva columna de prioridad:

- Se crea la columna **Prioridad** en la tabla **Pedidos** para permitir una clasificación adicional de los pedidos en función de su urgencia o importancia.

- Este cambio es útil cuando se quiere optimizar el proceso de atención a pedidos, permitiendo priorizar aquellos que requieren una acción más rápida.

2. Eliminación de la columna obsoleta:

- **Eliminar la columna EstadoEnvio** es una buena práctica si esa columna ya no es relevante o si se ha reemplazado por una mejor manera de gestionar el estado del envío, ya sea a través de un sistema de códigos o de una nueva columna.
- Mantener la base de datos limpia de columnas obsoletas mejora el rendimiento de las consultas y facilita el mantenimiento de la estructura de la base de datos.

Explicación de las buenas prácticas utilizadas en el proyecto:

1. Modificación estructural de tablas:

- **Agregar columnas** es un proceso común cuando se necesita almacenar nueva información relacionada con los datos existentes. Al agregar **Prioridad**, se está mejorando la capacidad de gestión de pedidos.
- **Eliminar columnas obsoletas** es una práctica fundamental para mantener la base de datos eficiente. Eliminar la columna **EstadoEnvio** previene la acumulación de datos innecesarios que puedan generar confusión o incrementar el tamaño de la base de datos sin un propósito claro.

2. Optimización de datos:

- **Gestionar prioridades** es una buena práctica en el manejo de pedidos, ya que permite a los empleados de ventas o logística identificar rápidamente cuáles pedidos deben recibir atención urgente.

3. Mantenimiento de la base de datos:

- La **limpieza y actualización** de la estructura de la base de datos es crucial para asegurar que la base de datos se mantenga optimizada, sin columnas obsoletas que puedan afectar el rendimiento o la claridad de los datos.

Proyecto 9: Implementación de registros automáticos de modificaciones en QhatuPeru

Enunciado del ejercicio:

Es necesario auditar todas las modificaciones realizadas en la tabla **Cientes** para cumplir con políticas internas de seguridad y protección de datos.

```
1  -- Crear la tabla AuditoriaClientes para registrar cambios en Clientes
2  CREATE TABLE AuditoriaClientes (
3      AuditoriaID INT IDENTITY(1,1) PRIMARY KEY,
4      ClienteID INT,
5      Nombre NVARCHAR(100),
6      Correo NVARCHAR(100),
7      Telefono NVARCHAR(15),
8      Accion NVARCHAR(10), -- 'INSERT', 'UPDATE', 'DELETE'
9      FechaModificacion DATETIME DEFAULT GETDATE(),
10     Usuario NVARCHAR(50)
11 );
12 GO
13
14 -- Crear un trigger para registrar eliminaciones en la tabla AuditoriaClientes
15 CREATE TRIGGER Trg_AuditoriaEliminarClientes
16 ON Clientes
17 INSTEAD OF DELETE
18 AS
19 BEGIN
20     -- Insertar los registros eliminados en la tabla AuditoriaClientes
21     INSERT INTO AuditoriaClientes (ClienteID, Nombre, Correo, Telefono, Accion, Usuario)
22     SELECT ClienteID, Nombre, Correo, Telefono, 'DELETE', SYSTEM_USER
23     FROM deleted;
24
25     -- Eliminar el registro de la tabla Clientes
26     DELETE FROM Clientes WHERE ClienteID IN (SELECT ClienteID FROM deleted);
27 END;
28 GO
29
```

Justificación técnica de la solución aplicada:

1. Auditoría de cambios en la base de datos:

- El **trigger** y la **tabla de auditoría** permiten mantener un registro de todas las modificaciones que se realizan en los datos sensibles de la tabla **Cientes**.
- Esto asegura que, en caso de problemas o revisiones, se pueda rastrear el **origen** de los cambios y la **identidad** del usuario que los realizó.

2. Uso de la pseudo-tabla deleted:

- La pseudo-tabla **deleted** permite capturar los valores de los registros que se van a eliminar, asegurando que se mantenga una copia de los datos antes de que se eliminen permanentemente.

3. Protección de datos:

- La **auditoría de eliminaciones** es una buena práctica para asegurar la **integridad** de los datos. Si se eliminan registros de forma

incorrecta o accidental, siempre se puede recuperar el historial de esas eliminaciones.

Explicación de las buenas prácticas utilizadas en el proyecto:

1. Uso de triggers para la auditoría:

- Utilizar triggers para auditar los cambios en las tablas garantiza que las modificaciones se registren de manera automática, lo que facilita el seguimiento de todas las acciones realizadas en los datos críticos.

2. Manejo de la integridad de los datos:

- Registrar las **acciones de eliminación** es crucial para prevenir la pérdida accidental de datos. Incluso si se produce un error humano, se tiene un registro de los cambios que se pueden revisar.

3. Automatización de la auditoría:

- La automatización de la auditoría mediante el uso de triggers elimina la necesidad de intervención manual para registrar las modificaciones, lo que mejora la eficiencia y asegura que los registros estén siempre actualizados.

Proyecto 10: Simulación de restauración tras un incidente en QhatuPeruNuevo

Enunciado del ejercicio:

Por un error humano se eliminó información de la tabla **Clientes**. El equipo debe restaurar la base usando el respaldo más reciente y validar la recuperación exitosa de los datos.

Simulación de eliminación de registros:

```
1  -- Simulación de eliminación de registros de Clientes
2  DELETE FROM Clientes WHERE ClienteID > 100;
3  GO
4
```



```
USE master;
GO

-- Restaurar la base de datos desde el respaldo más reciente
✓ RESTORE DATABASE QhatuPeruNuevo
FROM DISK = 'C:\QhatuPeru\Backups\QhatuPeruNuevo_Daily.bak'
WITH REPLACE;
GO
```

Ejercicio b) Verifica la existencia y correcta restauración de los datos con una consulta

```
-- Verificar la existencia de los registros restaurados
SELECT * FROM Clientes WHERE ClienteID > 100;
GO
```

Justificación técnica de la solución aplicada:

1. Simulación de pérdida de datos:

- **Simular la eliminación** de registros es útil para probar el proceso de recuperación. Nos permite verificar que el respaldo realizado anteriormente pueda recuperar datos perdidos debido a un error humano o un incidente.

2. Restauración desde respaldo:

- **RESTORE DATABASE** es el comando utilizado para recuperar la base de datos desde un archivo de respaldo. Al usar **WITH REPLACE**, garantizamos que la base de datos se sobrescriba con los datos del respaldo más reciente, asegurando la restauración completa de los datos.

3. Verificación de la restauración:

- **Verificar los datos restaurados** con una consulta asegura que la restauración se haya realizado correctamente. Es una buena práctica revisar que los registros eliminados hayan sido recuperados con éxito.

Explicación de las buenas prácticas utilizadas en el proyecto:

1. Importancia de los respaldos regulares:

- Los **respaldos diarios** son una práctica esencial para garantizar que se pueda recuperar la base de datos en caso de pérdida de datos. Los respaldos regulares aseguran que los datos estén protegidos contra errores humanos, fallos del sistema o incidentes inesperados.

2. **Uso de RESTORE DATABASE para la recuperación:**

- La restauración de la base de datos desde un archivo de respaldo es una de las formas más efectivas y confiables de recuperar datos perdidos. Esto permite restaurar todo el estado de la base de datos a un punto en el tiempo anterior al incidente.

3. **Verificación de la restauración:**

- Después de restaurar la base de datos, es fundamental **verificar** que los datos restaurados sean los correctos y que la restauración se haya realizado correctamente. Esto garantiza que el proceso de recuperación haya sido exitoso.